Jogo de sumô Kung Fu Panda — documento do projeto

Jogo em C para a disciplina de Estrutura de Dados · Grupo de 5 integrantes · Prazo de desenvolvimento: 1 mês · Entrega 01 (especificação): 25/10/2026, até 23:59

1. Visão geral

Item	Decisão
Filme	*Kung Fu Panda* (DreamWorks Animation, 2008)
Personagens	Personagens do filme (Po, Tai Lung e outros); a lista final ainda será definida
Gênero do jogo	Luta de sumô com física, para jogar em grupo, inspirada em *Sumotori Dreams* (corpos articulados que se equilibram e caem) e *Absolver* (combo montado pelo jogador)
Objetivo	Empurrar o adversário para fora do ringue
Jogadores	2 jogadores, cada um com um controle (gamepad); teclado como reserva
Mapa	Um único mapa: o ringue de sumô (dohyô), sem enfeites. O foco é a mecânica
Nome do jogo	Em aberto (sugestões: Kung Fu Sumô, Dohyô do Dragão, Skadoosh Sumô)

2. Tecnologia e visual

- **Linguagem:** C (padrão C99).
- **Interface gráfica 3D com raylib 6.0.** Todos os integrantes usam a mesma versão.
- **Física com ODE (Open Dynamics Engine)**, um motor de física 3D com API em C.
- **Corpo articulado, como no Sumotori:** cada lutador é formado por 9 peças físicas (bacia, tronco, cabeça, 2 braços, 2 coxas, 2 canelas) ligadas por articulações com motores.
- **Mola de equilíbrio:** uma força puxa o tronco continuamente para ficar de pé. O lutador fica de pé, mas balança, tropeça e cai quando leva uma pancada forte.
- **Modelos 3D gerados no Meshy AI e cortados em peças no Blender:** cada pedaço do personagem (cabeça, tronco, braços, pernas) é desenhado preso à sua peça física, como um boneco de ação articulado.
- **Câmera fixa e elevada**, olhando o ringue na diagonal.
- **Plano B** (se a verificação da semana 1 falhar, seção 7.9): física própria com uma cápsula por lutador e ragdoll só na queda.

Por que o corpo articulado? No Sumotori não existe uma hitbox separada: o próprio corpo é a física. Quando um braço empurra uma barriga, são duas peças se chocando de verdade. É isso que torna cada luta diferente e engraçada, e é o que o jogo quer reproduzir.

3. Controles

Comando	Ação
Analógico esquerdo	Andar (força aplicada na bacia do lutador)
Botão A	Executar o próximo golpe do combo
Gatilho	Firmar: a mola de equilíbrio fica mais forte e o lutador anda mais devagar

4. Mecânicas

4.1 Combo montado (núcleo da luta)

- Antes da luta, cada jogador monta um combo de **4 espaços** a partir de um catálogo de golpes: empurrão, levantador, tapa lateral, barrigada e investida.
- Na luta, cada aperto de A executa o golpe atual e avança para o próximo. Depois do último, volta ao primeiro.
- Ficar um tempo sem apertar **reinicia** o combo.
- Cada golpe é uma **pose**: os motores das articulações levam o corpo rapidamente para aquela posição (o braço estica, a barriga avança), e a peça que bate recebe um impulso.

4.2 Hematomas (dano)

- Um golpe **acerta quando a peça de ataque toca o corpo do adversário** (contato real, informado pelo ODE).
- Cada acerto cria um hematoma com **dano** e **tempo de cura**. Golpe leve: pouco dano e cura rápida. Golpe forte: muito dano e cura lenta.
- A **porcentagem** do lutador é a soma dos hematomas ativos.
- Quanto maior a porcentagem, **mais fraca fica a mola de equilíbrio** e **maior o impulso** dos golpes recebidos: o lutador machucado cambaleia mais e cai mais fácil.

4.3 Stamina

- Cada golpe gasta stamina, e ela regenera quando o jogador não ataca.
- Sem stamina, o jogador não ataca.

Juntas, essas mecânicas criam a tensão da luta: **atacar antes que o adversário se cure, sem perder o fôlego**.

5. Conteúdo da disciplina no jogo

Conteúdo	Onde	Justificativa
Lista circular duplamente encadeada	Combo (combo.c)	Circular porque o combo se repete: depois do último golpe vem o primeiro. Encadeada para inserir e remover golpes na montagem. Duplamente encadeada para mover golpes e voltar ao anterior.
Lista encadeada ordenada	Hematomas (hematoma.c)	Os hematomas entram e saem em momentos diferentes. A lista fica ordenada pelo momento de cura, então o jogo só verifica o primeiro elemento a cada quadro.
Insertion sort	Inserção de cada hematoma	Cada hematoma chega sozinho a uma lista que já está ordenada, que é o melhor caso do algoritmo (custo próximo de O(n)). É estável: hematomas com o mesmo tempo de cura mantêm a ordem de criação.

As estruturas de dados ficam acima da física: o combo decide qual golpe sai, e os hematomas decidem o quanto o lutador está fraco. A física só executa.

IA na dinâmica do jogo: não. A IA generativa (Meshy AI) é usada apenas para criar os modelos 3D, fora da execução do jogo.

6. Ferramentas

Critério: só entra o que resolve um problema real do projeto e cabe no prazo.

Usar

Ferramenta	Para que serve no projeto	Quem / quando
raylib 6.0	Janela, desenho 3D, modelos, controles e som.	Todos, desde o início
ODE	Física dos corpos articulados: peças, articulações com motores, colisões e contatos. Versão estável mais recente, a mesma nas 5 máquinas.	Física, semana 1
RayLibOdeRagDoll	Projeto de referência (jan/2026) que já junta raylib, ODE e ragdoll com articulações motorizadas, organizado em vários .c e sem variáveis globais. Ponto de partida para a semana 1.	Física, semana 1
CMake	Um único CMakeLists.txt que baixa e compila a raylib e o ODE igual nas 5 máquinas. Uma pessoa configura; os outros só compilam.	Integração, semana 1
Git + GitHub	Código compartilhado, histórico e uma branch por tarefa.	Todos, semana 1
Meshy AI	Gerar os modelos 3D dos personagens.	Arte, semana 1
Blender	Cortar cada modelo em 9 peças, acertar escala, orientação e ponto de origem de cada peça, reduzir polígonos e exportar em GLB.	Arte, semanas 1 e 2
rFXGen	Gerar efeitos sonoros (pancada, golpe, queda) em .wav.	Render e som, semana 3

Não usar

- **raygui:** a tela de montagem é simples e a raylib já desenha o necessário.
- **gltf-transform:** o Blender já faz o mesmo trabalho.
- **Jolt Physics e Box2D:** o ODE já cobre a física, com API em C e um projeto de referência com raylib.
- **Rig e animação esquelética do Meshy:** as peças rígidas substituem o esqueleto; quem anima o corpo é a física.

Polimento sem ferramenta nenhuma (semanas 3 e 4): pausa curta no impacto, tremor de câmera e porcentagem que pula ao subir.

7. Arquitetura

A arquitetura é a prioridade do projeto. Esta seção define as pastas, os módulos, os tipos e as funções antes de qualquer código. **O que estiver aqui é o contrato entre os 5 integrantes:** mudou um tipo ou uma função, atualiza este documento.

7.1 Princípios

- **Um módulo, uma responsabilidade.** Cada par .h/.c cuida de uma coisa só.
- **As estruturas de dados não dependem da raylib nem do ODE.** combo.c e hematoma.c são C puro: recebem o tempo como parâmetro. Assim podem ser testadas no terminal, sem janela.
- **Só fisica.c e corpo.c conhecem o ODE.** O resto do jogo fala com o corpo por funções como corpo_andar e corpo_aplicar_pose. Se a física precisar ser trocada (plano B), só esses dois módulos mudam.
- **A física roda em passo fixo** (FISICA_DT, por exemplo 1/120 s), independente da taxa de quadros. Isso evita que os corpos tremam ou explodam.
- **A lógica nunca desenha; o desenho nunca altera o estado.** As funções de render recebem ponteiros const.
- **Sem variáveis globais.** O estado vive em structs passadas por ponteiro.
- **Todo número ajustável fica em config.h**, com unidade e comentário.
- **Quem cria, destrói.** Cada módulo que aloca memória ou cria objetos do ODE tem a sua função de liberar.

7.2 Estrutura de pastas

```
jogo-sumo/
|-- CMakeLists.txt          compila o jogo; baixa e compila raylib 6.0 e ODE
|-- README.md               como compilar e rodar
|-- docs/                   este documento e anotações do grupo
|-- assets/
|   |-- modelos/<personagem>/   9 peças .glb por personagem + ringue
|   `-- sons/                   .wav gerados no rFXGen
|-- src/
|   |-- main.c              janela, laço principal e troca de telas
|   |-- config.h            todas as constantes ajustáveis
|   |-- esqueleto.h         peças, articulações e Pose (só tipos, C puro)
|   |-- golpe.h/.c          catálogo de golpes (dados fixos)
|   |-- personagem.h/.c     catálogo de personagens (dados fixos)
|   |-- combo.h/.c          LISTA CIRCULAR DUPLAMENTE ENCADEADA
|   |-- hematoma.h/.c       LISTA ORDENADA + INSERTION SORT
|   |-- fisica.h/.c         mundo do ODE, passo fixo e registro de contatos
|   |-- corpo.h/.c          corpo articulado: peças, motores, equilíbrio
|   |-- entrada.h/.c        controle/teclado -> Intencao
|   |-- lutador.h/.c        corpo + stamina + combo + hematomas
|   |-- combate.h/.c        contatos viram acertos: hematoma e impulso
|   |-- partida.h/.c        rounds, placar e regra de saída do ringue
|   |-- montagem.h/.c       tela de montagem do combo
|   |-- recursos.h/.c       carregar e liberar modelos e sons
|   `-- render.h/.c         câmera, peças dos personagens e HUD
`-- testes/
    |-- teste_combo.c       testes da lista circular (terminal)
    `-- teste_hematoma.c    testes da lista ordenada (terminal)
```

7.3 Dependências entre módulos

Cada módulo só pode incluir os que estão abaixo dele. Nenhum módulo inclui main.c ou render.

```
main.c
  |-- montagem ----------> combo, golpe, entrada
  |-- partida -----------> fisica, combate, lutador, config
  |     combate ---------> lutador, fisica, golpe, hematoma
  |     lutador ---------> corpo, combo, hematoma, golpe, personagem, entrada
  |     corpo -----------> fisica, esqueleto, personagem          [usa ODE]
  |     fisica ----------> esqueleto                              [usa ODE]
```

```
  |-- render (só lê) ----> partida, recursos, corpo
  `-- recursos ----------> personagem, esqueleto

C puro (sem raylib, sem ODE):  combo, hematoma, golpe, personagem, esqueleto
```

7.4 Convenções de código

O quê	Regra	Exemplo
Funções	snake_case, com o nome do módulo como prefixo	combo_avancar, corpo_aplicar_pose
Tipos (struct)	PascalCase com typedef	Combo, Corpo, Lutador
Enums	Tipo em PascalCase; valores em MAIÚSCULAS com prefixo	ParteCorpo: PARTE_TRONCO
Constantes	MAIÚSCULAS em config.h, com unidade no comentário	FISICA_DT /* s */
Unidades	Distância em metros, massa em kg, tempo em segundos, ângulo em radianos	float tempo_cura; /* s */
Comentários	Todo .h abre com um bloco dizendo o que o módulo faz. Toda função pública tem um comentário: o que faz, parâmetros, retorno e, nas estruturas de dados, a complexidade.	ver 7.5
Retorno de erro	Funções que podem falhar (malloc, criar corpo) retornam bool	bool combo_inserir(...)
Parâmetros só de leitura	Marcados com const	const Lutador *alvo

7.5 Tipos e funções por módulo

Assinaturas propostas. Os campos e nomes podem ser ajustados na semana 1, desde que este documento seja atualizado junto.

esqueleto.h — peças, articulações e pose (C puro)

```
typedef enum {
    PARTE_BACIA, PARTE_TRONCO, PARTE_CABECA,
    PARTE_BRACO_E, PARTE_BRACO_D,
    PARTE_COXA_E,  PARTE_COXA_D,
    PARTE_CANELA_E, PARTE_CANELA_D,
    PARTE_QTD                     /* 9 peças; sempre o último */
} ParteCorpo;

typedef enum {
    ART_CINTURA,                  /* bacia  - tronco  */
    ART_PESCOCO,                  /* tronco - cabeça  */
    ART_OMBRO_E, ART_OMBRO_D,     /* tronco - braço   */
    ART_QUADRIL_E, ART_QUADRIL_D, /* bacia  - coxa    */
    ART_JOELHO_E, ART_JOELHO_D,   /* coxa   - canela  */
    ART_QTD                       /* 8 articulações */
} Articulacao;

/* Ângulos que os motores tentam alcançar. Até 2 eixos por articulação. */
typedef struct {
    float alvo[ART_QTD][2];       /* (rad) */
} Pose;
```

golpe.h — catálogo de golpes (dados fixos, C puro)

```
typedef enum {
    GOLPE_EMPURRAO, GOLPE_LEVANTADOR, GOLPE_TAPA_LATERAL,
    GOLPE_BARRIGADA, GOLPE_INVESTIDA,
    GOLPE_QTD
} TipoGgolpe;

typedef struct {
    const char *nome;
    Pose  pose;               /* posição do corpo durante o golpe       */
    ParteCorpo parte_ataque;  /* peça que acerta (braço, tronco...)     */
    float impulso;            /* força aplicada no alvo ao acertar (N.s) */
    float impulso_vertical;   /* para cima, usado pelo levantador  (N.s) */
    float dano;               /* porcentagem somada ao alvo        (%)   */
    float tempo_cura;         /* quanto o hematoma dura            (s)   */
    float custo_stamina;      /*                                   (pts) */
    float duracao;            /* tempo em que a pose é mantida     (s)   */
} Golpe;

const Golpe *golpe_dados(TipoGolpe tipo);   /* catálogo constante */
```

personagem.h — catálogo de personagens (dados fixos, C puro)

```
typedef struct {
    const char *nome;              /* "Po", "Tai Lung", ...                */
    const char *pasta_modelos;     /* "assets/modelos/po/" (9 peças .glb)  */
    float tamanho[PARTE_QTD][3];   /* dimensões de cada peça física   (m)  */
    float massa[PARTE_QTD];        /* massa de cada peça              (kg) */
    float forca_equilibrio;        /* força da mola de equilíbrio     (N.m) */
    float forca_andar;             /* força aplicada na bacia ao andar (N) */
} Personagem;

int personagem_quantidade(void);
const Personagem *personagem_dados(int indice);
```

Adicionar um personagem novo é acrescentar uma linha no catálogo e uma pasta com as 9 peças. Nenhum outro módulo muda. O Po pode ter a bacia e o tronco mais pesados que o Tai Lung, por exemplo.

combo.h — LISTA CIRCULAR DUPLAMENTE ENCADEADA (C puro)

```
typedef struct NoCombo {
    TipoGolpe golpe;          /* guarda só o tipo; os dados vêm do catálogo */
    struct NoCombo *prox;     /* próximo; o último aponta para o primeiro   */
    struct NoCombo *ant;      /* anterior; o primeiro aponta para o último  */
} NoCombo;

typedef struct {
    NoCombo *inicio;          /* primeiro golpe (NULL se vazio)            */
    NoCombo *atual;           /* golpe que sai no próximo aperto de A      */
    int tamanho;              /* quantidade de golpes (máx. COMBO_MAX)     */
} Combo;

void      combo_iniciar(Combo *c);                        /* combo vazio */
bool      combo_inserir(Combo *c, int pos, TipoGolpe g);  /* O(n) */
bool      combo_remover(Combo *c, int pos);               /* O(n) */
bool      combo_trocar_com_proximo(Combo *c, int pos);    /* O(n) */
TipoGolpe combo_golpe_atual(const Combo *c);              /* O(1) */
void      combo_avancar(Combo *c);     /* atual = atual->prox, O(1) */
void      combo_reiniciar(Combo *c);   /* atual = inicio,      O(1) */
void      combo_liberar(Combo *c);     /* free em todos os nós, O(n) */
```

hematoma.h — LISTA ORDENADA + INSERTION SORT (C puro)

```
typedef struct NoHematoma {
    float  dano;              /* porcentagem deste hematoma   (%) */
    double momento_cura;      /* instante em que ele some     (s) */
    struct NoHematoma *prox;
} NoHematoma;

typedef struct {
    NoHematoma *inicio;       /* o que cura primeiro                   */
    int   quantidade;
    float dano_total;         /* soma mantida a cada inserção e remoção */
} ListaHematomas;

void  hematomas_iniciar(ListaHematomas *l);

/* Cria o hematoma e o insere na posição certa pelo momento_cura
   (insertion sort). Retorna false se faltar memória.  O(n) */
bool  hematoma_inserir(ListaHematomas *l, float dano, double momento_cura);

/* Remove do início todos os hematomas com momento_cura <= agora.
   Retorna quantos removeu.  O(k), k = removidos */
int   hematomas_curar(ListaHematomas *l, double agora);

float hematomas_dano_total(const ListaHematomas *l);   /* O(1) */
void  hematomas_liberar(ListaHematomas *l);            /* O(n) */
```

fisica.h — mundo do ODE (usa ODE)

```
/* Identifica uma peça: guardada como "user data" em cada forma do ODE,
   para saber de quem é a peça quando duas se tocam. */
typedef struct {
    int lutador;              /* 0 ou 1; -1 para o chão */
    ParteCorpo parte;
} RefPeca;

/* Toque entre peças de lutadores diferentes, registrado no passo. */
typedef struct {
    RefPeca a, b;
    float intensidade;        /* velocidade relativa no toque (m/s) */
} Contato;

typedef struct {
    dWorldID      mundo;
    dSpaceID      espaco;
    dJointGroupID grupo_contatos;   /* contatos temporários do ODE */
    dGeomID       chao;
    Contato contatos[FISICA_MAX_CONTATOS];
    int     qtd_contatos;           /* zerado no início de cada passo */
    float   acumulador;             /* tempo ainda não simulado (s)   */
} Fisica;

bool fisica_iniciar(Fisica *f);             /* cria mundo, gravidade, chão */
/* Avança a simulação em passos fixos de FISICA_DT até consumir dt. */
void fisica_atualizar(Fisica *f, float dt);
void fisica_liberar(Fisica *f);
```

corpo.h — corpo articulado (usa ODE)

```
typedef struct {
    dBodyID  pecas[PARTE_QTD];        /* corpos rígidos                  */
    dGeomID  formas[PARTE_QTD];       /* formas de colisão (caixa/cápsula) */
    dJointID articulacoes[ART_QTD];   /* articulações com motor          */
    RefPeca  refs[PARTE_QTD];         /* "user data" de cada forma       */
    Pose     pose_alvo;               /* para onde os motores puxam      */
    float    equilibrio;              /* multiplicador da mola, 0 a 1    */
} Corpo;

/* Cria as 9 peças e 8 articulações em pé, na posição dada. */
bool    corpo_criar(Corpo *c, Fisica *f, const Personagem *p,
                    int indice_lutador, Vector3 posicao);
void    corpo_aplicar_pose(Corpo *c, const Pose *pose);   /* troca o alvo   */
void    corpo_pose_neutra(Corpo *c);                      /* volta ao normal */
/* A cada passo: motores em direção à pose e mola de equilíbrio
   no tronco, com força forca_equilibrio * equilibrio. */
void    corpo_atualizar(Corpo *c, const Personagem *p, float dt);
void    corpo_andar(Corpo *c, const Personagem *p, Vector2 direcao);
void    corpo_impulso(Corpo *c, ParteCorpo parte, Vector3 impulso);
Vector3 corpo_posicao(const Corpo *c);          /* centro da bacia        */
Vector3 corpo_frente(const Corpo *c);           /* para onde está virado  */
bool    corpo_caido(const Corpo *c);            /* tronco quase deitado   */
Matrix  corpo_transformacao(const Corpo *c, ParteCorpo parte);  /* render */
void    corpo_destruir(Corpo *c);
```

entrada.h — controle e teclado

```
typedef struct {
    Vector2 mover;            /* direção do analógico, de -1 a 1 em x e y */
    bool    golpear;          /* botão A apertado neste quadro          */
    bool    firmar;           /* gatilho segurado                       */
} Intencao;

/* Lê o controle do jogador (0 ou 1). Sem controle, usa o teclado.
   Aplica a zona morta do analógico (ENTRADA_ZONA_MORTA). */
Intencao entrada_ler(int jogador);
```

lutador.h — um lutador

```
typedef enum {
    LUTADOR_NORMAL, LUTADOR_GOLPEANDO, LUTADOR_FIRMADO,
    LUTADOR_CAIDO,            /* no chão, a mola tenta levantar */
    LUTADOR_FORA              /* saiu do ringue */
} EstadoLutador;

typedef struct {
    int personagem;           /* índice no catálogo de personagens     */
    int indice;               /* 0 ou 1                                */
    Corpo corpo;              /* peças físicas                         */
    float stamina;            /* de 0 a STAMINA_MAX              (pts) */
    EstadoLutador estado;
    float tempo_no_estado;    /*                                 (s)   */
    float tempo_sem_golpear;  /* para reiniciar o combo          (s)   */
    const Golpe *golpe_atual; /* NULL se não está golpeando            */
    bool  golpe_ja_acertou;   /* um golpe acerta no máximo uma vez     */
    Combo combo;              /* lista circular do jogador             */
    ListaHematomas hematomas; /* lista ordenada de hematomas recebidos */
} Lutador;

bool  lutador_criar(Lutador *l, Fisica *f, int personagem, int indice,
                    Vector3 posicao);
void  lutador_aplicar_intencao(Lutador *l, Intencao in);  /* andar, firmar */
bool  lutador_pode_golpear(const Lutador *l);             /* estado, stamina */
/* Gasta stamina, aplica a pose do golpe e chama combo_avancar. */
const Golpe *lutador_iniciar_golpe(Lutador *l);
/* Stamina, hematomas_curar, equilíbrio a partir do dano total,
   fim do golpe, reinício do combo, detecção de queda. */
void  lutador_atualizar(Lutador *l, float dt, double agora);
void  lutador_destruir(Lutador *l);   /* corpo + as 2 listas */
```

combate.h — contatos viram acertos

```
/* Percorre os contatos do passo. Se a peça de ataque de um lutador
   que está golpeando tocou o outro, aplica o golpe (uma vez só). */
void combate_processar(Lutador lut[2], const Fisica *f, double agora);

/* Insere o hematoma no alvo (insertion sort) e aplica o impulso,
   que cresce com o dano total do alvo. */
void combate_aplicar_golpe(Lutador *atacante, Lutador *alvo,
                           ParteCorpo parte_atingida, double agora);
```

partida.h — rounds, placar e regras

```
typedef enum {
    PARTIDA_CONTAGEM,         /* 3, 2, 1 antes do round */
    PARTIDA_LUTA,
    PARTIDA_FIM_ROUND,
    PARTIDA_FIM
} EstadoPartida;

typedef struct {
    Fisica  fisica;           /* um mundo do ODE por partida */
    Lutador lutadores[2];
    int vitorias[2];
    int round;
    EstadoPartida estado;
    float tempo_no_estado;    /* (s) */
} Partida;

bool partida_iniciar(Partida *p, int personagem1, int personagem2);
void partida_atualizar(Partida *p, Intencao in[2], float dt, double agora);
bool partida_fora_do_ringue(const Lutador *l);  /* bacia além do raio */
int  partida_vencedor(const Partida *p);        /* -1 se não acabou   */
void partida_liberar(Partida *p);
```

montagem.h, recursos.h, render.h e main.c

```
/* montagem: tela onde cada jogador monta o seu combo */
typedef struct { int cursor[2]; bool pronto[2]; } TelaMontagem;
void montagem_atualizar(TelaMontagem *t, Combo combos[2], Intencao in[2]);
bool montagem_concluida(const TelaMontagem *t);

/* recursos: tudo que vem de arquivo */
typedef struct {
    Model pecas[MAX_PERSONAGENS][PARTE_QTD];  /* 9 modelos por personagem */
    Model ringue;
    Sound sons[SOM_QTD];
} Recursos;
bool recursos_carregar(Recursos *r);
void recursos_liberar(Recursos *r);

/* render: só lê o estado (const) e desenha cada peça com
   corpo_transformacao(...) */
void render_luta(const Partida *p, const Recursos *r);
void render_hud(const Partida *p);
void render_montagem(const TelaMontagem *t, const Combo combos[2]);

/* main.c: troca de telas */
typedef enum { TELA_MENU, TELA_MONTAGEM, TELA_LUTA, TELA_RESULTADO } Tela;
```

7.6 O que acontece em um quadro da luta

```
1. main      in[i] = entrada_ler(i)                 para os 2 jogadores
2. partida   lutador_aplicar_intencao(...)          corpo_andar, firmar
3. partida   se in.golpear e lutador_pode_golpear:
                 lutador_iniciar_golpe(...)         stamina, pose, combo_avancar
4. partida   lutador_atualizar(...)                 hematomas_curar, equilíbrio
                                                    = 1 - dano/DANO_MAX, stamina,
                                                    combo_reiniciar, queda
5. partida   fisica_atualizar(...)                  passos fixos do ODE:
                                                    motores, mola, colisões,
                                                    registro de contatos
6. partida   combate_processar(...)                 contato da peça de ataque
                                                    -> hematoma_inserir
                                                       (insertion sort) + impulso
7. partida   partida_fora_do_ringue(...)            fim do round?
8. main      render_luta(...) e render_hud(...)     só desenha
```

7.7 Memória e objetos do ODE

- Só duas estruturas usam malloc: os nós do combo e os nós dos hematomas. combo_remover e hematomas_curar liberam o nó na hora em que ele sai da lista.
- Os objetos do ODE (peças, formas, articulações) são criados em corpo_criar e destruídos em corpo_destruir. O mundo e o espaço são criados em fisica_iniciar e destruídos em fisica_liberar.
- lutador_destruir chama corpo_destruir, combo_liberar e hematomas_liberar. partida_liberar destrói os dois lutadores e depois a física. main chama dInitODE ao abrir o jogo e dCloseODE ao fechar.
- Modelos e sons são carregados uma vez (recursos_carregar) e liberados ao fechar o jogo.

7.8 Testes

Como combo.c e hematoma.c não dependem da raylib nem do ODE, cada um tem um programa de teste no terminal (pasta testes/), usando assert:

- **Combo:** inserir 4 golpes e avançar 5 vezes (o 5º deve ser o 1º de novo); remover o único nó; remover o nó atual; trocar o último com o primeiro; liberar e verificar que ficou vazio.
- **Hematomas:** inserir fora de ordem (8 s, 3 s, 5 s) e conferir a ordem 3, 5, 8; dois com o mesmo tempo mantêm a ordem de criação; curar no tempo certo e conferir o dano total; curar com a lista vazia.

Esses testes também servem de demonstração para a professora.

7.9 Verificação da semana 1 (física)

O corpo articulado é o maior risco do projeto. No fim da semana 1, o grupo confere se estes 5 itens funcionam:

#	Critério
1	raylib e ODE compilam pelo CMake nas 5 máquinas.
2	Um corpo de 9 peças (desenhadas como caixas) fica de pé sozinho por 30 segundos com a mola de equilíbrio, sem tremer nem explodir.
3	O corpo anda pelo ringue com o analógico.
4	Um impulso forte derruba o corpo, e a mola tenta levantá-lo.
5	Uma pose de golpe (braço esticado) é aplicada pelos motores e o contato com outro corpo é registrado.

Se os 5 funcionarem: o projeto segue com o corpo articulado. **Se não:** entra o plano B: uma cápsula por lutador com física própria (andar, empurrão, choque e queda) e o ODE só para o ragdoll da queda, ou nem isso. Como só fisica.c e corpo.c conhecem o ODE, a troca fica restrita a esses dois módulos: combo, hematomas, golpes, partida e render continuam iguais.

8. Divisão de tarefas por módulo

Integrante	Módulos	Entrega da semana 1
1. Física	fisica, corpo, esqueleto	Itens 2 a 5 da verificação (seção 7.9), a partir do RayLibOdeRagDoll
2. Estruturas de dados	combo, hematoma, testes/	Listas funcionando e testadas no terminal
3. Arte	assets/, valores de golpe e personagem	Po do Meshy cortado em 9 peças e exportado
4. Render e interface	render, recursos, montagem	Câmera, ringue e peças do Po desenhadas
5. Integração	main, partida, lutador, combate, entrada, CMake, Git	Item 1 da verificação, controles lidos, apoio à física

Todos precisam saber explicar o combo (lista circular) e os hematomas (insertion sort).

9. Ideias descartadas

Fardos na borda do ringue, chuva de bolinhos, marcas e orbes, ordenação do ranking e do catálogo de golpes, modo torneio, replay, partículas, 4 jogadores, fila circular de entradas e física própria com cilindros (virou o plano B).

10. Pontos em aberto

Para resolver antes da entrega de 25/10

- Nome do jogo.
- Bibliografia: usar os livros indicados pela professora.
- Revisar o texto dos itens a a j do formulário (a seção 2 mudou: física com ODE e corpo articulado) e escolher quem envia.

Decisões de design (semana 1)

- Lista final de personagens.
- Forma de cada peça física (caixa ou cápsula) e as proporções de cada personagem.
- Valores de cada golpe (pose, impulso, dano, tempo de cura, custo de stamina, duração).
- Força da mola de equilíbrio e quanto o dano a enfraquece.
- Cair no chão dentro do ringue também perde o round, como no sumô de verdade?
- Regra dos rounds: melhor de 3? Hematomas e stamina zeram a cada round?
- Tela de montagem: o combo fica salvo entre partidas?

Técnico e organização

- Fazer a verificação da semana 1 (seção 7.9). É o maior risco do projeto.
- Testar o corte do modelo do Meshy em peças no Blender com o Po.
- Definir os modelos de controle (Xbox ou compatível é o mais garantido no Windows).
- Confirmar quem fica com cada módulo (seção 8) e criar o repositório Git.